



Deep Learning

Tutorial 9: Object Detection using SSD & Yolo

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

- Use the data you developed in previous tutorial and test it on the models developed or if you can improve them by adding and removing layers
- Take one pretrained model each of Yolo and SSD to train and test on your data

Implementation of these tasks are in PyTorch:

The screenshot displays a Jupyter Notebook interface with a dark theme. The top bar shows the file name 'Mubashar_494171_Tutorial_9.ipynb' and various menu options like File, Edit, View, Insert, Runtime, Tools, and Help. The left sidebar shows a file explorer with a folder named 'Mubashar Tutorial 9' containing a subfolder 'runs' and a file 'detect'. The main area is divided into two sections: 'Install Dependencies' and 'Import Libraries'.

Install Dependencies

```
!pip install roboflow ultralytics torchvision
```

The output shows a list of requirements already satisfied, including:

- Requirement already satisfied: fsspec<0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (0.8.5)
- Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.9.4)
- Requirement already satisfied: nvidia-cuda-mrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.8.93)
- Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.8.90)
- Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.8.90)
- Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (9.10.2.21)
- Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.8.4.1)
- Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (11.3.3.83)
- Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (10.3.9.90)
- Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (11.7.3.90)
- Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.5.8.93)
- Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.5.8.93)
- Requirement already satisfied: nvidia-mccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (2.27.5)
- Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (3.4.5)
- Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.8.90)
- Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>1.8.0-ultralytics) (12.8.93)

Import Libraries

```
import torch
import torchvision
import torchvision.transforms as T
from torch import nn
from torch.utils.data import DataLoader
import os
```

Download Dataset (Roboflow)

```
from roboflow import Roboflow

rf = Roboflow(api_key="2yhE0XoVP8vJERSiNyYa")
project = rf.workspace("mubashar-workspace-nd2nw").project("wildfire-vxrin-sende")
version = project.version(1)
dataset = version.download("yolov11")
```

The output shows the following progress:

```
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in wildfire-1 to yolov11: 100% [██████████] 115672/115672 [00:08<00:00, 14458.94it/s]

Extracting Dataset Version Zip to wildfire-1 in yolov11: 100% [██████████] 1439/1439 [00:00<00:00, 2946.41it/s]
```

Check Dataset Path

[4]

Os

dataset.location

▼

▼

YOLov11 Implementation (Ultralytics)

Train YOLOv11

[5]

4m

```

from ultralytics import YOLO

# Load pretrained YOLOv8 nano model as YOLOv11 is not directly supported.
# You can replace 'yolov8n.pt' with other supported YOLOv8 models (e.g., 'yolov8s.pt', 'yolov8m.pt') if needed.
model = YOLO("yolov8n.pt")

# Train
model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=20,
    imgsz=640,
    batch=16
)

```

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
19/20	2.4G	1.873	1.958	1.702	39	640: 100%	32/32	3.1it/s 10.4s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 2.8it/s 1.8s
	all	143	756	0.38	0.348	0.319	0.12	

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
20/20	2.4G	1.86	1.938	1.687	34	640: 100%	32/32	3.4it/s 9.4s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	5/5 2.1it/s 2.3s
	all	143	756	0.388	0.374	0.325	0.124	

20 epochs completed in 0.073 hours.

Optimizer stripped from /content/runs/detect/train/weights/last.pt, 6.2MB

Optimizer stripped from /content/runs/detect/train/weights/best.pt, 6.2MB

Validating /content/runs/detect/train/weights/best.pt...

Ultralytics 8.4.51 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	
all	143	756	0.39	0.371	0.324	0.124	5/5 1.8it/s 2.8s
Fire	133	533	0.364	0.505	0.385	0.15	
smoke	88	223	0.415	0.238	0.263	0.0967	

Speed: 0.5ms preprocess, 3.3ms inference, 0.0ms loss, 6.6ms postprocess per image

Results saved to /content/runs/detect/train

ultralytics.utils.metrics.DetMetrics object with attributes:

3

```
Validation

[6] ✓ 5s
model.val()

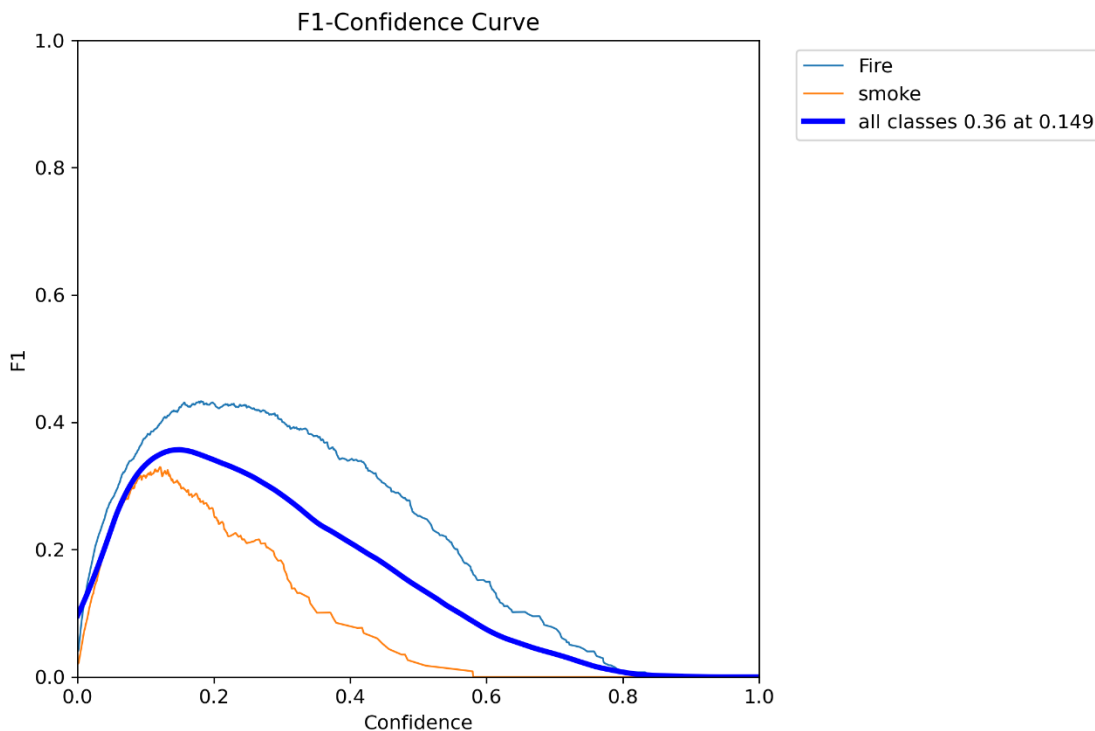
Ultralytics 8.4.51 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 2245.6±1084.0 MB/s, size: 105.5 KB)
val: Scanning /content/wildfire-1/valid/labels.cache... 143 images, 1 backgrounds, 0 corrupt: 100% 143/143 50.0Mit/s 0.0s
      Class      Images  Instances  Box(P)      R      mAP50  mAP50-95): 100% 9/9 3.0it/s 3.0s
      all         143         756      0.393      0.378      0.325      0.124
      Fire        133         533      0.357      0.505      0.383      0.15
      smoke        88         223      0.428      0.251      0.266      0.0985

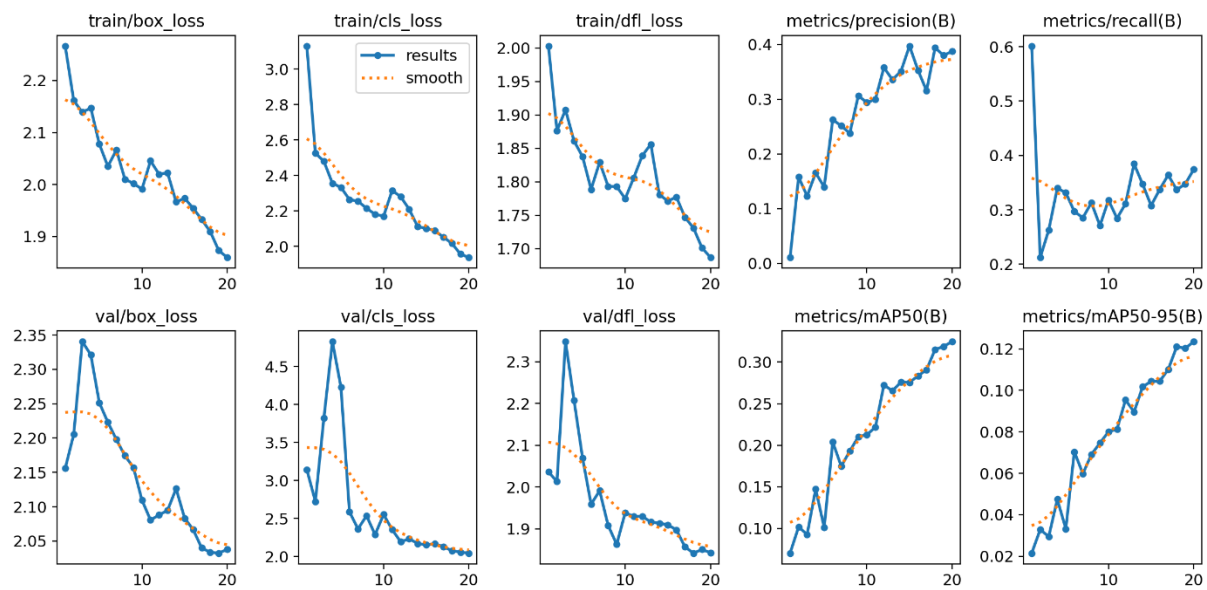
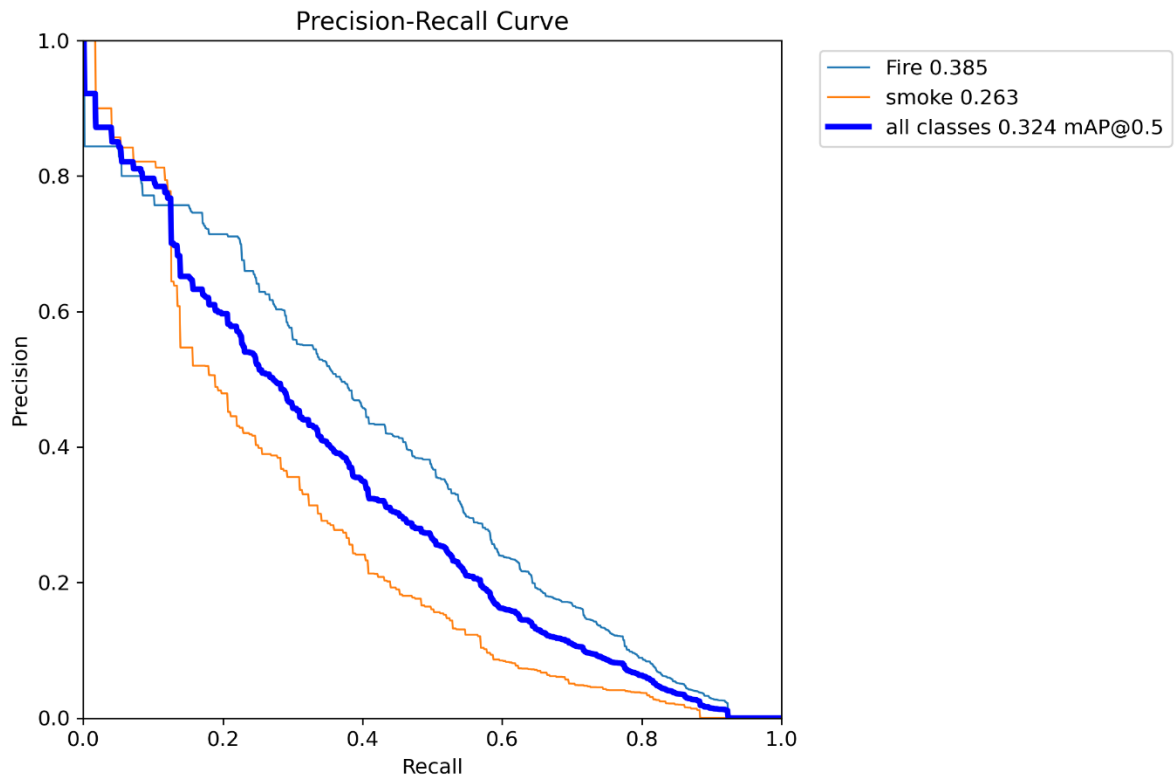
Speed: 3.7ms preprocess, 4.6ms inference, 0.0ms loss, 2.6ms postprocess per image
Results saved to /content/runs/detect/val
ultralytics.utils.metrics.DetMetrics object with attributes:

Inference

[7] ✓ 2s
results = model.predict(
    source=f"{dataset.location}/test/images",
    save=True
)

image 1/72 /content/wildfire-1/test/images/fire-101_png.rf.545bd2d75f72d14ed99eb8343cbecf8d.jpg: 480x640 2 Fires, 39.2ms
image 2/72 /content/wildfire-1/test/images/fire-108_png.rf.b09657b1498fa5645cacc84bc16def26.jpg: 448x640 1 Fire, 39.4ms
image 3/72 /content/wildfire-1/test/images/fire-119_png.rf.17b1547edd263e96407ca097033e7002.jpg: 512x640 3 Fires, 38.7ms
image 4/72 /content/wildfire-1/test/images/fire-125_png.rf.bbb7a09d4da4a935b3d0f4c47feea5fdb.jpg: 352x640 5 Fires, 39.1ms
image 5/72 /content/wildfire-1/test/images/fire-134_png.rf.898f7244d85383941f0a9095a5358115.jpg: 384x640 3 Fires, 40.5ms
image 6/72 /content/wildfire-1/test/images/fire-141_png.rf.6ae4345ff3dfdb9a8df791efc89cee.jpg: 640x416 2 Fires, 42.5ms
image 7/72 /content/wildfire-1/test/images/fire-146_png.rf.4b00ea686090b9bc03cbe03f3776d06a.jpg: 416x640 3 Fires, 39.8ms
image 8/72 /content/wildfire-1/test/images/fire-167_png.rf.51baf98f7ba4cc723e5dec75d64a4d0.jpg: 384x640 2 Fires, 7.1ms
image 9/72 /content/wildfire-1/test/images/fire-169_png.rf.a5b84ebc3c459bc702d1c1499e6252cb.jpg: 384x640 1 Fire, 5.8ms
image 10/72 /content/wildfire-1/test/images/fire-190_png.rf.387f4bc3bb8fd78a5f70c25312c1ab12.jpg: 480x640 3 Fires, 1 smoke, 6.9ms
image 11/72 /content/wildfire-1/test/images/fire-194_png.rf.d384773512c3ee2b4e778b241e291b46.jpg: 384x640 3 Fires, 6.6ms
image 12/72 /content/wildfire-1/test/images/fire-195_png.rf.20dac853904e6c27b4865573587896b7.jpg: 384x640 3 Fires, 6.0ms
image 13/72 /content/wildfire-1/test/images/fire-19_png.rf.b90b34d020eafd35a6238f4f1b123577.jpg: 320x640 1 Fire, 1 smoke, 50.5ms
image 14/72 /content/wildfire-1/test/images/fire-1_png.rf.0070fbf259470f4864d068429a7f2fbc.jpg: 352x640 6 Fires, 6.5ms
image 15/72 /content/wildfire-1/test/images/fire-203_png.rf.09f68f1c4adf8ae62119ff5901cbffc2.jpg: 384x640 3 Fires, 7.9ms
image 16/72 /content/wildfire-1/test/images/fire-208_png.rf.683b1208ccdeb36851b65b05852757a.jpg: 384x640 3 Fires, 5.7ms
image 17/72 /content/wildfire-1/test/images/fire-20_png.rf.9e56b25e16b648488fa01a291c720196.jpg: 416x640 3 Fires, 1 smoke, 6.3ms
```







Custom SSD (From Scratch)

Define Backbone

```
[8]
✓ Os
class Backbone(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

    def forward(self, x):
        return self.features(x)
```

SSD Prediction Head

```
[9]
✓ Os
class SSDHead(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.loc = nn.Conv2d(128, 4 * 4, kernel_size=3, padding=1) # 4 boxes
        self.cls = nn.Conv2d(128, num_classes * 4, kernel_size=3, padding=1)

    def forward(self, x):
        loc = self.loc(x)
        cls = self.cls(x)
        return loc, cls
```

Full SSD Model

```
[10]
✓ Os
class SSD(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.backbone = Backbone()
        self.head = SSDHead(num_classes)

    def forward(self, x):
        features = self.backbone(x)
        loc, cls = self.head(features)
        return loc, cls
```

Test SSD with Dummy Data

[+ Code](#)[+ Text](#)

```
[11]
✓ 0s
model = SSD(num_classes=2) # wildfire dataset → adjust if needed

dummy = torch.randn(1, 3, 224, 224)
loc, cls = model(dummy)

print("Localization shape:", loc.shape)
print("Classification shape:", cls.shape)
```

```
Localization shape: torch.Size([1, 16, 28, 28])
Classification shape: torch.Size([1, 8, 28, 28])
```

Pretrained SSD (Torchvision)

Load Pretrained SSD

```
[12]
✓ 3s
from torchvision.models.detection import ssd300_vgg16

model = ssd300_vgg16(pretrained=True)
model.eval()

(5): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(6): ReLU(inplace=True)
(7): Sequential(
  (0): MaxPool2d(kernel_size=3, stride=1, padding=1, dilation=1, ceil_mode=False)
  (1): Conv2d(512, 1024, kernel_size=(3, 3), stride=(1, 1), padding=(6, 6), dilation=(6, 6))
  (2): ReLU(inplace=True)
  (3): Conv2d(1024, 1024, kernel_size=(1, 1), stride=(1, 1))
  (4): ReLU(inplace=True)
)
(1): Sequential(
  (0): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1))
  (1): ReLU(inplace=True)
```


Modify for Custom Classes

```
[14]
✓ 0s
from torchvision.models.detection.ssd import SSDClassificationHead, SSDRegressionHead

# Define the number of custom classes (Fire, Smoke)
num_classes = 2

# Define the actual number of classes including background (num_classes from previous cell is 2 for Fire, Smoke)
num_classes_with_background = num_classes + 1 # 2 custom classes + 1 for background = 3

# Extract original input channels for the classification and regression heads
in_channels_list = [layer.in_channels for layer in model.head.classification_head.module_list]

# Extract the number of anchors per feature map from the regression head
# The output channels of regression head layers are num_anchors * 4 (for bbox coordinates)
num_anchors_per_feature_map = [layer.out_channels // 4 for layer in model.head.regression_head.module_list]

# Create new classification and regression heads for the custom number of classes
new_classification_head = SSDClassificationHead(
    in_channels_list,
    num_anchors_per_feature_map,
    num_classes_with_background
)
new_regression_head = SSDRegressionHead(
    in_channels_list,
    num_anchors_per_feature_map
)

# Replace the existing heads in the model with the newly created ones
model.head.classification_head = new_classification_head
model.head.regression_head = new_regression_head

print(f"SSD model adapted for {num_classes_with_background} classes (2 custom classes + background).")

SSD model adapted for 3 classes (2 custom classes + background).
```

Simple Training Loop (Skeleton)

```
[15]
✓ 26m
from torchvision.datasets import VOCDetection
from torch.utils.data import DataLoader

def custom_collate_fn(batch):
    images = []
    targets = []
    for item in batch:
        img_tensor, voc_target = item # img_tensor is already a tensor due to dataset transform

        # Apply ToTensor transform to the image (removed, as it's already applied by the dataset's transform)
        # img_tensor = T.ToTensor()(img_pil)
        images.append(img_tensor)

        # Process VOC target to model-compatible format
        boxes = []
        labels = []

        # VOCDetection target['annotation']['object'] can be a dict or a list of dicts
        objects = voc_target['annotation']['object']
        if not isinstance(objects, list):
            objects = [objects]

        for obj in objects:
            bndbox = obj['bndbox']
            xmin = float(bndbox['xmin'])
            ymin = float(bndbox['ymin'])
            xmax = float(bndbox['xmax'])
            ymax = float(bndbox['ymax'])
            boxes.append([xmin, ymin, xmax, ymax])
```

```

        # Assign a dummy label for now.
        # In a real scenario, you would map VOC class names to your custom labels.
        # Assuming 'Fire' is class 1, 'smoke' is class 2. Background is 0.
        # Since this is VOC, let's map all detected objects to class 1 (arbitrary choice).
        labels.append(1) # All VOC objects will be treated as 'Fire' for training

    if not boxes:
        # Handle images with no annotations by creating empty tensors
        boxes_tensor = torch.zeros((0, 4), dtype=torch.float32)
        labels_tensor = torch.zeros((0,), dtype=torch.int64)
    else:
        boxes_tensor = torch.as_tensor(boxes, dtype=torch.float32)
        labels_tensor = torch.as_tensor(labels, dtype=torch.int64)

    targets.append({'boxes': boxes_tensor, 'labels': labels_tensor})

return images, targets

transform = T.Compose([
    T.ToTensor()
])

dataset = VOCDetection(
    root="./data",
    year="2012",
    image_set="train",
    download=True,
    transform=transform
)

dataloader = DataLoader(dataset, batch_size=4, shuffle=True, collate_fn=custom_collate_fn)

# Move model to GPU
model.to("cuda")

# Set model to training mode
model.train()

optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(5):
    for images, targets in dataloader:
        images = list(img.to("cuda") for img in images)
        targets = [{k: v.to("cuda") for k, v in t.items()} for t in targets]

        loss_dict = model(images, targets)
        loss = sum(loss for loss in loss_dict.values())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch} Loss: {loss.item()}")

```

```

... 100%|██████████| 2.00G/2.00G [05:24<00:00, 6.16MB/s]
Epoch 0 Loss: 1.5329923629760742
Epoch 1 Loss: 1.136224627494812
Epoch 2 Loss: 1.5505263805389404
Epoch 3 Loss: 2.5253734588623047
Epoch 4 Loss: 0.44960957765579224

```

Dataset Loader for SSD

[16]
✓ 19s



```
from torchvision.datasets import VOCDetection

transform = T.Compose([
    T.ToTensor()
])

dataset = VOCDetection(
    root="./data",
    year="2012",
    image_set="train",
    download=True,
    transform=transform
)

dataloader = DataLoader(dataset, batch_size=4, shuffle=True)
```